

Q.1 Determine the time complexity:

```
for(int i=0;i<n;i++)  
{  
    cout<<i;  
}
```

Answer: $O(n)$

- Loop runs n times

Q.2 Determine the time complexity:

```
for(int i=0;i<n;i++)  
{  
    for(int j=0;j<n;j++)  
    {  
        cout<<"*";  
    }  
}
```

Answer: $O(n^2)$

- $n \times n$ iterations

Q.3 Determine the time complexity:

```
for(int i=0;i<n;i++)  
{  
    for(int j=0;j<i;j++)  
    {  
        cout<<"*";  
    }  
}
```

Answer: $O(n^2)$

- $1+2+\dots+(n-1) = n(n-1)/2$

Q.4 Determine the time complexity:

```
for(int i=1;i<n;i*=2)  
{  
    cout<<i;  
}
```

Answer: $O(\log n)$

- i doubles each time

Q.5 Determine the time complexity:

```
for(int i=n;i>1;i/=2)
{
    cout<<i;
}
```

Answer: $O(\log n)$

- i halves each time

Q.6 Determine the time complexity:

```
for(int i=1;i<=n;i++)
{
    for(int j=1;j<=log(n);j++)
    {
        cout<<"*";
    }
}
```

Answer: $O(n \log n)$

- Outer n , inner $\log n$

Q.7 Determine the time complexity:

```
while(n>0)
{
    n--;
}
```

Answer: $O(n)$

- n decrements to 0

Q.8 Determine the time complexity:

```
while(n>1)
{
    n/=2;
}
```

Answer: $O(\log n)$

- n divided by 2 repeatedly

Q.9 Determine the time complexity:

```

for(int i=1;i<=n;i++)
{
}
for(int j=1;j<=n;j++)
{
}

```

Answer: $O(n)$

- Two separate $O(n)$ loops $\Rightarrow O(2n) = O(n)$

Q.10 Determine the time complexity:

```

for(int i=1;i<=n;i++)
{
}
for(int j=1;j<=n*n;j++)
{
}

```

Answer: $O(n^2)$

- $O(n) + O(n^2) = O(n^2)$

Q.11 Determine the time complexity:

```

for(int i=1;i<=n;i++)
{
    for(int j=1;j<=n*n;j++)
    {
    }
}

```

Answer: $O(n^3)$

- $n \times n^2$

Q.12 Determine the time complexity:

```

for(int i=1;i<=n;i*=3)
{
}

```

Answer: $O(\log_3 n) = O(\log n)$

- i multiplies by 3

Q.13 Determine the time complexity:

```
for(int i=1;i<=n;i++)
{
    for(int j=1;j<=i*i;j++)
    {
    }
}
```

Answer: $O(n^3)$

- $\sum i^2 = n(n+1)(2n+1)/6$

Q.14 Determine the time complexity:

```
void fun(int n)
{
    if(n==0)
        return;

    fun(n-1);
}
```

Answer: $O(n)$

- One recursive call, depth n

Q.15 Determine the time complexity:

```
void fun(int n)
{
    if(n<=1)
        return;

    fun(n/2);
}
```

Answer: $O(\log n)$

- One recursive call, size halves

Q.16 Determine the time complexity:

```
void fun(int n)
{
    if(n==0)
        return;
```

```

    fun(n-1);
    fun(n-1);
}

```

Answer: $O(2^n)$

- $T(n) = 2T(n-1) + 1$

Q.17 Determine the time complexity:

```

int fib(int n)
{
    if(n<=1)
        return n;

    return fib(n-1)+fib(n-2);
}

```

Answer: $O(2^n)$

- Naive Fibonacci recursion

Q.18 Determine the time complexity:

```

int fact(int n)
{
    if(n<=1)
        return 1;

    return n*fact(n-1);
}

```

Answer: $O(n)$

- One recursive call per level

Q.19 Determine the time complexity:

```

void fun(int n)
{
    if(n<=0)
        return;

    fun(n-1);

    cout<<n;
}

```

```
}
```

Answer: $O(n)$

- Recursive depth n

Q.20 Determine the time complexity:

```
void fun(int n)
```

```
{
```

```
    if(n<=1)
```

```
        return;
```

```
    fun(n/2);
```

```
    cout<<n;
```

```
}
```

Answer: $O(\log n)$

- Recursive depth $\log n$

Q.21 Solve the recurrence relation:

$$T(n) = T(n-1) + 1$$

Answer: $O(n)$

- $T(n) = T(n-1) + 1$

Q.22 Solve the recurrence relation:

$$T(n) = T(n-1) + n$$

Answer: $O(n^2)$

- $1 + 2 + \dots + n$

Q.23 Solve the recurrence relation:

$$T(n) = T(n/2) + 1$$

Answer: $O(\log n)$

- Halving recurrence

Q.24 Solve the recurrence relation:

$$T(n) = 2T(n-1) + 1$$

Answer: $O(2^n)$

- Expands as binary recursion

Q.25 Solve the recurrence relation:

$$T(n) = T(n/2) + n$$

Answer: $O(n)$

- Master Theorem: $a=1, b=2, f(n)=n$

Q.26 Determine the complexity.

```
for(int i=1;i<=n;i++)
{
    for(int j=1;j<=i;j*=2)
    {
    }
}
```

Answer: $O(n \log n)$

- Outer loop n , inner loop $\log i$

Q.27 Determine the complexity.

```
for(int i=1;i<=n;i*=2)
{
    for(int j=1;j<=n;j++)
    {
    }
}
```

Answer: $O(n \log n)$

- Outer = $O(\log n)$, Inner = $O(n)$

Q.28 Determine the complexity.

```
for(int i=n;i>=1;i--)
{
    for(int j=i;j>=1;j--)
    {
    }
}
```

Answer: $O(n^2)$

- $n+(n-1)+\dots+1 = n(n+1)/2$

Q.29 Determine the complexity.

```
for(int i=1;i<=n;i++)
{
    for(int j=1;j<=n;j*=2)
    {
    }
}
```

Answer: $O(n \log n)$

- Outer = $O(n)$, Inner = $O(\log n)$

Q.30 Determine the complexity.

```
void fun(int n)
{
    if(n<=1)
        return;

    fun(n/2);
    fun(n/2);
}
```

Answer: $O(n)$

- $a = 2$
- $b = 2$
- $n^{\log_b a} = n$